

COMP 356
Programming Language Structures
Notes for Chapter 8 of *Concepts of Programming Languages*
Control Structures

Definitions:

- a **control statement** is any statement that alters the linear flow of control of a program. C++ examples include: **if-else**, **while**, **for**, **break**, **continue** and **return** statements.
- a **control structure** is a control statement along with the other statements that it controls

Some programmers prefer **single-entry, single-exit** control structures in which the flow of control can only enter a control structure at one point, and also can only leave a control structure at one point. For example, a Java **if** statement produces a single-entry, single-exit control structure if the structure does not contain any **break** or **return** statements (or throw any exceptions).

Typically, multiple entries to a control structure can only appear in a language with **goto**'s (C, C++, Ada – but not Java). Such structures are extremely harmful to readability and maintainability of code.

Most “standard” imperative languages provide multiple exits from control structures via **break**, **return**, **goto** or more specialized statements. In general, these are not harmful to readability (except for **goto**) unless overused.

1 Selection Statements

A **selection statement** is a control statement that allows choosing between two or more execution paths in a program. Most modern programming languages include one-way, two-way and n-way (multiple) selectors.

- an **if** statement with no **else** is a one-way selector
- an **if-else** statement is a two-way selector
- a **switch** or **case** statement is an n-way selector

All selection can be implemented using only a one-way selector, but the additional forms increase the readability and writeability of programs.

Nested two-way selectors introduce a design problem, because the following grammar:

```
<stmt> → if (<expr>) <stmt>
        | if (<expr>) <stmt> else <stmt>
        | other
```

is ambiguous, as shown by the parse trees for the string:

```
if (<expr>)
  if (<expr>) <stmt>
else <stmt>
```

- in C, C++, C# and Java, an **else** is always matched with the nearest unmatched **if**, so the indentation above is misleading.
- in a YACC or CUP parser, the ambiguity can be resolved (and this matching behavior forced) by giving the token for **else** highest precedence.
- in a C program, the programmer can specify the matching desired by using blocks appropriately. For example:

```
if (<expr>) {
    if (<expr>) <stmt>
} else <stmt>
```

Compound statements:

- were originally designed to make control structures simpler
- transform a sequence of statements into a single logical statement, so no extra control statement syntax is required for sequences of statements
- were later extended to blocks by allowing the declaration of local variables
- in other languages (Perl, Ada, VB), a special closing word such as **end if** is required to end every if statement. This resolves the problem and reduces the potential for programmer error.

In C, C++ and Java, the n-way selector is **switch**. Each **case** must end with a **break** statement if following cases should not be executed. The ability to execute multiple cases is sometimes convenient, but is rarely used and forgetting the **break** statements is a common source of errors.

In most other programming languages, only one case in an n-way selection statement can be executed when the selection statement is executed. Many languages also allow more elaborate comparisons between the **selector value** (the value of the expression following the keyword **switch**) and the values labeling each case.

- in C, C++ and Java, the case labels can only be constants, and the selector value and the case labels can only be compared for equality.
- in Ada, case labels can also be subranges, and then the comparison is whether or not the selector value occurs in the subrange.
- in Visual Basic, case labels can be constants, subranges and variables, and can use arbitrary relational operators (using the keyword **Is** to represent the selector value).

Almost all languages allow a **default** case that is executed if the selector value does not match the label(s) of any other case.

- some languages require the case labels to be exhaustive (have a match for all possible selector values)
- a **default** case is often convenient for error reporting etc. in languages that do not require the labels to be exhaustive
- if the case labels are not exhaustive and the selector value does not match any case, typically the n-way selection statement has no effect

2 Loops

Definitions:

- the **body** of a loop is the collection of statements controlled by the loop
- a **pretest** loop is a loop in which the loop condition is tested before execution of the body – for example, a **while** loop in C, C++ and Java.
- a **posttest** loop is a loop in which the loop condition is tested after execution of the body – for example, a **do-while** loop in C, C++ and Java.

Many languages also have control statements that can be used in the body and that affect the execution of the loop:

- in C, C++ and Java, a **break** statement in a loop body causes execution to jump to the first statement after the loop
- in Ada, the **exit** statement is similar to **break**. There is also a conditional form (**exit when**).

- in C, C++ and Java, a `continue` statement in a loop body causes execution to jump to the control point of the loop (the “top” of a pretest loop and the “bottom” of a posttest loop), skipping the remainder of the loop body

Note that all of these statements are restricted forms of `goto`’s.

In C, C++ and Java the `for` loop can be used for both counting loops and for logically controlled (`while`) loops. For example, the following `for` loop in C reads characters from standard input:

```
char ch;
for (ch = getchar(); ch != EOF; ch = getchar()) {
    /* process ch here */
}
```

The handling of the loop control variable in `for` loops varies greatly by language, specifically in whether the variable can be changed (assigned to) in the loop body and what value it has after the loop terminates:

- in Pascal, the loop control variable can not be assigned to in the loop body and is undefined after loop termination
- in Ada, the loop control variable is implicitly declared in the `for` statement. Implications:
 - the loop control variable should NOT be explicitly declared
 - the implicit declaration shadows any other declaration of a variable with the same name during the execution of the loop

The loop control variable can not be assigned to in the loop body.

- in C, C++ and Java, the loop control variable receives no special protection, and its value is accessible after loop termination if it is in scope. In C++ and Java, the loop control variable can be declared in the initialization expression, and its scope is then the body of the loop (except for older versions of Visual C++).

In Pascal and Ada, the initial and terminal values of the loop control variable can not be changed during loop execution, even if those values are given using variables and those variables are assigned to in the loop body.

Some languages include loop statements for processing each element of a “collection” data structure. For example, Perl and Tcl have a `foreach` loop that processes each element of a list or array. Other languages include iterators (i.e. for the Java `Collection` interface). These are not a new control structure, but rather data structures and methods that are often used in an ordinary loop.

3 Unconditional Branching

The unconditional branch (`goto`) statement is the most powerful control statement – all control structures can be built using only `goto` and a one-way selector. The use of `goto` means that any statement in a program can potentially be executed immediately after any other statement, and so is quite harmful both to readability and maintainability. Perhaps the most famous paper in computer science history is “Goto Statement Considered Harmful” (Dijkstra, 1968).

Because the efficiency advantages of `gotos` can sometimes outweigh the dangers of using them, almost all “standard” programming languages include them. Note that Java does not have a `goto`. Statements such as `break`, `continue`, `exit` and `return` are restricted forms of `goto`. However, all of these statements attempt to do less harm to readability than `goto` by restricting the potential targets of the branch.